Join Exercise Questions part-1

==========

Employees / Departments / Projects

```
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   name VARCHAR(100),
   department_id INT,
   salary DECIMAL(10,2)
);

CREATE TABLE Departments (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(100)
);

CREATE TABLE Projects (
   project_id INT PRIMARY KEY,
   project_name VARCHAR(100),
   emp_id INT
);

INSERT INTO Departments (department_id, department_name) VALUES
(1, 'Human Resources'), (2, 'Engineering'), (3, 'Marketing'), (4, 'Finance'), (5,
'Operations'),(6,'HR');

INSERT INTO Employees (emp_id, name, department_id, salary) VALUES
(101, 'Arjun', 1, 50000), (102, 'Bhavana', 2, 65000), (103, 'Charan', 2, 72000),
```

(104, 'Divya', 3, 45000), (105, 'Eswar', NULL, 40000), (106, 'Farhan', 4, 55000),

(107, 'Gowri', 2, 62000), (108, 'Harish', 5, 48000), (109, 'Ishita', NULL, 53000),

(110, 'Jagan', 3, 60000);

INSERT INTO Projects (project_id, project_name, emp_id) VALUES

(201, 'Website Redesign', 102), (202, 'Mobile Banking App', 103), (203, 'Marketing Campaign', 104),

(204, 'Payroll System', 106), (205, 'Backend Optimization', 107), (206, 'Cloud Migration', 102),

(207, 'Customer Analytics', 110), (208, 'Inventory System', NULL);

Key facts this data is built to exercise:

- **Eswar and Ishita have department_id = NULL — employees with no department.**
- **Arjun, Harish have no project. Bhavana has *two* projects.**
- **Project 208 "Inventory System" has emp_id = NULL — a project with no one assigned.**
- **Every department has at least one employee, so no department is ever "empty" in this dataset (unlike the Students set below — see Part 2).**

INNER JOIN

Q1. Get all employees with their department names.

```
SELECT e.emp_id, e.name, d.department_name
FROM Employees e
INNER JOIN Departments d ON e.department_id = d.department_id;
```

emp_id	name	department_name
101	Arjun	Human Resources
102	Bhavana	Engineering

103	Charan	Engineering
104	Divya	Marketing
106	Farhan	Finance
107	Gowri	Engineering
108	Harish	Operations
110	Jagan	Marketing

8 rows — Eswar and Ishita are correctly dropped (NULL department, no match).

Q2. List all employees who are assigned to a project.

```
SELECT DISTINCT e.emp_id, e.name
FROM Employees e
INNER JOIN Projects p ON e.emp_id = p.emp_id;
```

emp_id	name
102	Bhavana
103	Charan
104	Divya
106	Farhan
107	Gowri
110	Jagan

Q3. Show employee names and their project names.

```
SELECT e.name, p.project_name
FROM Employees e
```

INNER JOIN Projects p ON e.emp_id = p.emp_id;

name	project_name
Bhavana	Website Redesign
Charan	Mobile Banking App
Divya	Marketing Campaign
Farhan	Payroll System
Gowri	Backend Optimization
Bhavana	Cloud Migration
Jagan	Customer Analytics

7 rows (no DISTINCT here, so Bhavana correctly appears twice for her two projects).

LEFT JOIN

Q1. All employees and their department names (even if no department).

SELECT e.emp_id, e.name, d.department_name

FROM Employees e

LEFT JOIN Departments d ON e.department_id = d.department_id;

emp_id	name	department_name
101	Arjun	Human Resources
102	Bhavana	Engineering
103	Charan	Engineering
104	Divya	Marketing

105	Eswar	*NULL*
106	Farhan	Finance
107	Gowri	Engineering
108	Harish	Operations
109	Ishita	*NULL*
110	Jagan	Marketing

Q2. All employees and their project names (if assigned).

```
SELECT e.name, p.project_name
FROM Employees e
LEFT JOIN Projects p ON e.emp_id = p.emp_id;
```

name	project_name
Arjun	*NULL*
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Eswar	*NULL*
Farhan	Payroll System
Gowri	Backend Optimization
Harish	*NULL*
Ishita	*NULL*

Jagan	Customer Analytics

11 rows — Arjun, Eswar, Harish, Ishita show NULL (no project); Bhavana still gets 2 rows.

Q3. All departments and the employees working in them.

```
SELECT d.department_name, e.name
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id;
```

department_name	name
Human Resources	Arjun
Engineering	Bhavana
Engineering	Charan
Engineering	Gowri
Marketing	Divya
Marketing	Jagan
Finance	Farhan
Operations	Harish

(No NULL rows here — every department in this dataset has at least one employee.)

RIGHT JOIN

Q1. All departments and the employees in them (even if none).

```
SELECT D.DEPARTMENT_NAME,E.NAME
   FROM DEPARTMENTS D
   RIGHT JOIN EMPLOYEES E
```

ON D.DEPARTMENT_ID=E.DEPARTMENT_ID;

Same 8 rows as LEFT JOIN Q3 above — because every department happens to have an employee, RIGHT JOIN here produces no visible NULLs. (LEFT and RIGHT only look different when there's actually an unmatched row on the "preserved" side.)

Q2. All projects and their assigned employees (even if none).

```
SELECT P.PROJECT_NAME,E.NAME
  FROM PROJECTS P
   RIGHT JOIN EMPLOYEES E
  -ON P.EMP_ID=E.EMP_ID;
```

8 rows — Inventory System correctly shows `NULL` for its unassigned employee. This is the row that actually proves the RIGHT JOIN is doing something LEFT JOIN wouldn't.

Q3. Employees and departments using RIGHT JOIN.

```
SELECT e.name, d.department_name
FROM Departments d
RIGHT JOIN Employees e ON d.department_id = e.department_id;
```

name	department_name
Arjun	Human Resources
Bhavana	Engineering
Charan	Engineering
Gowri	Engineering
Divya	Marketing
Jagan	Marketing

Farhan	**Finance**
Harish	**Operations**
Eswar	***NULL***
Ishita	***NULL***

10 rows — Eswar and Ishita show up with NULL department, since Employees is on the "preserved" (right) side here.

SELF JOIN

Q1. Employees who share the same department.

SELECT e1.name AS Employee1, e2.name AS Employee2, e1.department_id

FROM Employees e1

JOIN Employees e2 ON e1.department_id = e2.department_id AND e1.emp_id <> e2.emp_id;

Employee1	Employee2	department_id
Bhavana	**Charan**	**2**
Bhavana	**Gowri**	**2**
Charan	**Bhavana**	**2**
Charan	**Gowri**	**2**
Divya	**Jagan**	**3**
Gowri	**Bhavana**	**2**
Gowri	**Charan**	**2**
Jagan	**Divya**	**3**

Note: this returns both directions of each pair (Bhavana↔Charan appears twice). That's expected from <> — if you only want each pair once, use e1.emp_id < e2.emp_id instead, which would cut this to 4 rows.

Q2. Pairs of employees where one salary is more than the other.

```
SELECT e1.name AS Higher_Salary_Employee, e2.name AS Lower_Salary_Employee,
e1.salary, e2.salary

FROM Employees e1

JOIN Employees e2 ON e1.salary > e2.salary;
```

This returns 45 rows — every possible higher/lower salary pairing among all 10 employees (since all 10 salaries happen to be distinct, this is the full C(10,2) = 45 combinations). It's not wrong, but it's a lot of noise for an exercise answer — worth being aware the result set explodes quickly as employee count grows. (Full 45-row output available on request; omitted here for length — first few rows: Arjun > Divya, Arjun > Eswar, Arjun > Harish, Bhavana > Arjun, Bhavana > Divya…)

Q3. Employee names and colleagues from the same department.

```
SELECT e1.name AS Employee, e2.name AS Colleague, d.department_name

FROM Employees e1

JOIN Employees e2 ON e1.department_id = e2.department_id AND e1.emp_id <>
e2.emp_id

JOIN Departments d ON e1.department_id = d.department_id;
```

Employee	Colleague	department_name
Bhavana	Charan	Engineering
Bhavana	Gowri	Engineering
Charan	Bhavana	Engineering
Charan	Gowri	Engineering
Divya	Jagan	Marketing

Gowri	Bhavana	Engineering
Gowri	Charan	Engineering
Jagan	Divya	Marketing

FULL JOIN (simulated via UNION — MySQL has no native FULL JOIN)

Q1. All employees and all departments (even if no match).

SELECT e.name, d.department_name FROM Employees e LEFT JOIN Departments d ON e.department_id = d.department_id

UNION

SELECT e.name, d.department_name FROM Employees e RIGHT JOIN Departments d ON e.department_id = d.department_id;

name	department_name
Arjun	Human Resources
Bhavana	Engineering
Charan	Engineering
Divya	Marketing
Eswar	*NULL*
Farhan	Finance
Gowri	Engineering
Harish	Operations
Ishita	*NULL*
Jagan	Marketing

10 rows — every employee, including Eswar/Ishita with NULL department. (No department-side NULL rows appear because every department has at least one employee.)

Q2. All projects and employees (even if some don't match).

SELECT e.name, p.project_name FROM Employees e LEFT JOIN Projects p ON e.emp_id = p.emp_id

UNION

SELECT e.name, p.project_name FROM Employees e RIGHT JOIN Projects p ON e.emp_id = p.emp_id;

name	project_name
NULL	Inventory System
Arjun	*NULL*
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Eswar	*NULL*
Farhan	Payroll System
Gowri	Backend Optimization
Harish	*NULL*
Ishita	*NULL*
Jagan	Customer Analytics

12 rows — this is the one query in Part 1 that shows NULLs on both sides at once: Inventory System has no employee, and Arjun/Eswar/Harish/Ishita have no project.

Q3. All department names and employee names, with or without a match.

```
SELECT d.department_name, e.name FROM Departments d LEFT JOIN Employees e ON d.department_id = e.department_id

UNION

SELECT d.department_name, e.name FROM Departments d RIGHT JOIN Employees e ON d.department_id = e.department_id;
```

department_name	name
NULL	Eswar
NULL	Ishita
Engineering	Bhavana
Engineering	Charan
Engineering	Gowri
Finance	Farhan
Human Resources	Arjun
Marketing	Divya
Marketing	Jagan
Operations	Harish

JOIN + GROUP BY

Q1. Count employees per department.

```
SELECT d.department_name, COUNT(e.emp_id) AS Total_Employees

FROM Departments d

LEFT JOIN Employees e ON d.department_id = e.department_id

GROUP BY d.department_name;
```

department_name	Total_Employees
Engineering	3
Finance	1
Human Resources	1
Marketing	2
Operations	1

Q2. Total salary per department.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	Total_Salary
Engineering	199000
Finance	55000
Human Resources	50000
Marketing	105000
Operations	48000

Q3. Number of projects per employee.

```
SELECT e.name, COUNT(p.project_id) AS Total_Projects
FROM Employees e
LEFT JOIN Projects p ON e.emp_id = p.emp_id
```

GROUP BY e.name;

name	Total_Projects
Arjun	0
Bhavana	2
Charan	1
Divya	1
Eswar	0
Farhan	1
Gowri	1
Harish	0
Ishita	0
Jagan	1

JOIN + ORDER BY

Q1. Employees with department names, ordered by salary (highest first).

SELECT e.name, d.department_name, e.salary

FROM Employees e

JOIN Departments d ON e.department_id = d.department_id

ORDER BY e.salary DESC;

name	department_name	salary
Charan	Engineering	72000

Bhavana	Engineering	65000
Gowri	Engineering	62000
Jagan	Marketing	60000
Farhan	Finance	55000
Arjun	Human Resources	50000
Harish	Operations	48000
Divya	Marketing	45000

(Eswar/Ishita excluded — plain JOIN behaves as INNER JOIN.)

Q2. Employee names and project names, ordered by employee name.

```
SELECT e.name, p.project_name
FROM Employees e
JOIN Projects p ON e.emp_id = p.emp_id
ORDER BY e.name;
```

name	project_name
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Farhan	Payroll System
Gowri	Backend Optimization
Jagan	Customer Analytics

Q3. Departments and employee count, ordered by count (highest first).

```
SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY Employee_Count DESC;
```

department_name	Employee_Count
Engineering	3
Marketing	2
Operations	1
Human Resources	1
Finance	1

JOIN + GROUP BY + ORDER BY + HAVING

Q1. Departments with more than 1 employee, ordered by count.

```
SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING COUNT(e.emp_id) > 1
ORDER BY Employee_Count DESC;
```

department_name	Employee_Count

Engineering	3
Marketing	2

Q2. Employees with more than 1 project, ordered by count.

```
SELECT e.name, COUNT(p.project_id) AS Project_Count
FROM Employees e
JOIN Projects p ON e.emp_id = p.emp_id
GROUP BY e.name
HAVING COUNT(p.project_id) > 1
ORDER BY Project_Count DESC;
```

name	Project_Count
Bhavana	2

Q3. Departments with total salary > 50000, ordered by total salary.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING SUM(e.salary) > 50000
ORDER BY Total_Salary DESC;
```

department_name	Total_Salary
Engineering	199000
Marketing	105000

Finance	55000

(Human Resources = 50000 and Operations = 48000 are correctly excluded — neither is strictly greater than 50000.)

Joins With Clauses

==========

JOIN + GROUP BY

Q1. Count employees per department.

```
SELECT d.department_name, COUNT(e.emp_id) AS Total_Employees
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	Total_Employees
Engineering	3
Finance	1
Human Resources	1
Marketing	2
Operations	1

Q2. Total salary per department.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	Total_Salary
Engineering	199000
Finance	55000
Human Resources	50000
Marketing	105000
Operations	48000

Q3. Number of projects per employee.

```
SELECT e.name, COUNT(p.project_id) AS Total_Projects
FROM Employees e
LEFT JOIN Projects p ON e.emp_id = p.emp_id
GROUP BY e.name;
```

name	Total_Projects
Arjun	0
Bhavana	2
Charan	1
Divya	1
Eswar	0
Farhan	1
Gowri	1
Harish	0

Ishita	0
Jagan	1

JOIN + ORDER BY

Q1. Employees with department names, ordered by salary (highest first).

```
SELECT e.name, d.department_name, e.salary
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
ORDER BY e.salary DESC;
```

name	department_name	salary
Charan	Engineering	72000
Bhavana	Engineering	65000
Gowri	Engineering	62000
Jagan	Marketing	60000
Farhan	Finance	55000
Arjun	Human Resources	50000
Harish	Operations	48000
Divya	Marketing	45000

(Eswar/Ishita excluded — plain JOIN behaves as INNER JOIN.)

Q2. Employee names and project names, ordered by employee name.

```
SELECT e.name, p.project_name
FROM Employees e
```

JOIN Projects p ON e.emp_id = p.emp_id

ORDER BY e.name;

name	project_name
Bhavana	Website Redesign
Bhavana	Cloud Migration
Charan	Mobile Banking App
Divya	Marketing Campaign
Farhan	Payroll System
Gowri	Backend Optimization
Jagan	Customer Analytics

Q3. Departments and employee count, ordered by count (highest first).

SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count

FROM Departments d

LEFT JOIN Employees e ON d.department_id = e.department_id

GROUP BY d.department_name

ORDER BY Employee_Count DESC;

department_name	Employee_Count
Engineering	3
Marketing	2
Operations	1
Human Resources	1

Finance	1

JOIN + GROUP BY + ORDER BY + HAVING

Q1. Departments with more than 1 employee, ordered by count.

```
SELECT d.department_name, COUNT(e.emp_id) AS Employee_Count
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING COUNT(e.emp_id) > 1
ORDER BY Employee_Count DESC;
```

department_name	Employee_Count
Engineering	3
Marketing	2

Q2. Employees with more than 1 project, ordered by count.

```
SELECT e.name, COUNT(p.project_id) AS Project_Count
FROM Employees e
JOIN Projects p ON e.emp_id = p.emp_id
GROUP BY e.name
HAVING COUNT(p.project_id) > 1
ORDER BY Project_Count DESC;
```

name	Project_Count
Bhavana	2

Q3. Departments with total salary > 50000, ordered by total salary.

```
SELECT d.department_name, SUM(e.salary) AS Total_Salary
FROM Departments d
JOIN Employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING SUM(e.salary) > 50000
ORDER BY Total_Salary DESC;
```

department_name	Total_Salary
Engineering	199000
Marketing	105000
Finance	55000

(Human Resources = 50000 and Operations = 48000 are correctly excluded — neither is strictly greater than 50000.)